

//Selectoin sort

```
#include <iostream>
#include <vector>
using namespace std;

void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        swap(arr[i], arr[minIndex]);
    }
}
```

```
int main() {
    vector<int> arr = {64, 25, 12, 22, 11};
    selectionSort(arr);
    for (int i : arr) cout << i << " ";
    cout << endl;
}
```



11 12 22 25 64

//Bubble sort

```
#include <iostream>
#include <vector>
using namespace std;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}
```

```
}
```

```
int main() {  
    vector<int> arr = {64, 25, 12, 22, 11};  
    bubbleSort(arr);  
    for (int i : arr) cout << i << " ";  
    cout << endl;  
}
```



11 12 22 25 64

//Insertion sort

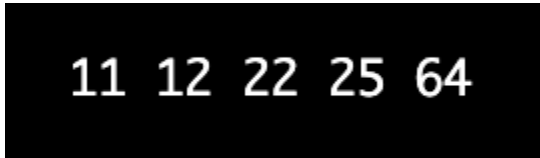
```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
void insertionSort(vector<int>& arr) {  
    int n = arr.size();  
    for (int i = 1; i < n; i++) {  
        int key = arr[i];  
        int j = i - 1;  
        while (j >= 0 && arr[j] > key) {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
        arr[j + 1] = key;  
    }  
}
```

```
int main() {  
    vector<int> arr = {64, 25, 12, 22, 11};  
    insertionSort(arr);  
    for (int i : arr) cout << i << " ";  
    cout << endl;  
}
```



11 12 22 25 64

//Right pivot quicksort

```
#include <iostream>
```

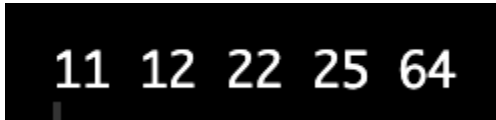
```
#include <vector>
```

```
using namespace std;
```

```
int part(vector<int>& arr, int low, int high) {  
    int pivot = arr[high];  
    int i = low - 1;  
    for (int j = low; j < high; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            swap(arr[i], arr[j]);  
        }  
    }  
    swap(arr[i + 1], arr[high]);  
    return i + 1;  
}
```

```
void quickSort(vector<int>& arr, int low, int high) {  
    if (low < high) {  
        int pi = part(arr, low, high);  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}
```

```
int main() {  
    vector<int> arr = {64, 25, 12, 22, 11};  
    quickSort(arr, 0, arr.size() - 1);  
    for (int i : arr) cout << i << " ";  
    cout << endl;  
}
```



11 12 22 25 64

//Mergesort

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
void merge(vector<int>& arr, int left, int mid, int right) {  
    int n1 = mid - left + 1;  
    int n2 = right - mid;  
    vector<int> L(n1), R(n2);  
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];  
    for (int i = 0; i < n2; i++) R[i] = arr[mid + 1 + i];
```

```

int i = 0, j = 0, k = left;
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) arr[k++] = L[i++];
    else arr[k++] = R[j++];
}
while (i < n1) arr[k++] = L[i++];
while (j < n2) arr[k++] = R[j++];
}

void mergeSort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};
    mergeSort(arr, 0, arr.size() - 1);
    for (int i : arr) cout << i << " ";
    cout << endl;
}

```



//Quicksort rand pivot

```

#include <iostream>
#include <vector>
#include <cstdlib>
using namespace std;

int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
}

```

```

    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

int randomizedPartition(vector<int>& arr, int low, int high) {
    int randomPivotIndex = low + rand() % (high - low + 1);
    swap(arr[randomPivotIndex], arr[high]);
    return partition(arr, low, high);
}

void randomizedQuickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = randomizedPartition(arr, low, high);
        randomizedQuickSort(arr, low, pi - 1);
        randomizedQuickSort(arr, pi + 1, high);
    }
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};
    randomizedQuickSort(arr, 0, arr.size() - 1);
    for (int i : arr) cout << i << " ";
    cout << endl;
    return 0;
}

```

11 12 22 25 64